National University of Computer and Emerging Sciences

Lab Manual 07
Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 7

Objectives:

- ✓ Genetic Algorithm
  - Population Generation
  - Fitness Evaluation
  - Selection, Crossover, and Mutation
  - Optimization using Genetic Algorithms

Activities:

Activity 1 (Genetic Algorithm Concepts):

Genetic Algorithm

A Genetic Algorithm is a search and optimization technique inspired by the principles of natural selection and genetics. It is commonly used to find approximate solutions to complex optimization and search problems where traditional methods may be inefficient.

Genetic Algorithms belong to a class of algorithms known as evolutionary algorithms, which simulate the process of biological evolution. They apply mechanisms like those found in nature, such as:

- Selection
- Crossover (Recombination)
- Mutation
- Inheritance

These mechanisms help the algorithm evolve better solutions over multiple generations.

Genetic Algorithms are considered global search heuristics, meaning they explore a large search space to find optimal or near-optimal solutions.

Working Principle of Genetic Algorithms

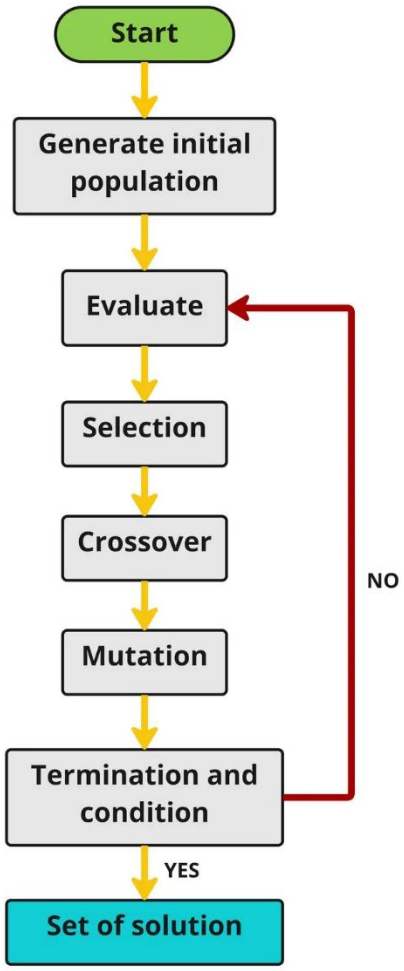
The basic idea of a Genetic Algorithm is to start with a population of possible solutions and improve them over time using evolutionary operations.

The algorithm works as follows:

- The process begins with a population of randomly generated individuals, where each individual represents a possible solution to the problem.
- In every generation, the fitness of each individual is evaluated using a fitness function that measures how good the solution is.
- Based on their fitness values, better individuals are selected from the population.
- Selected individuals undergo genetic operations such as crossover and mutation to produce a new generation of solutions.
- The newly generated population replaces the old population and the process repeats.
- The algorithm continues until one of the following conditions is met:
  - A maximum number of generations is reached, or
  - A solution with satisfactory fitness is found.

Genetic Algorithm Flow

The Genetic Algorithm (GA) follows an iterative process inspired by biological evolution. It repeatedly improves a population of candidate solutions until an optimal or satisfactory solution is found.



Step-by-Step Process

- *Initial Population*
  The algorithm begins by generating an initial population of individuals (candidate solutions). These individuals are usually created randomly.
- *Subset Input Selection (Selection)*
  From the current population, a subset of individuals is selected based on their fitness values. Individuals with higher fitness have a higher probability of being selected for reproduction.
- *Fitness Function Evaluation*
  Each individual is evaluated using a fitness function, which measures how good a solution is with respect to the target problem.
- *Best Chromosome Selection*
  The individuals with the highest fitness values are chosen as parents for producing the next generation.

- *Crossover and Mutation*
  - Crossover:
    Combines genetic information from two parent chromosomes to create new offspring.
  - Mutation:
    Introduces small random changes in genes to maintain diversity in the population and prevent premature convergence.
- *Stop Condition Check*
  The algorithm checks whether the termination condition has been reached. Common stopping criteria include:
  - Maximum number of generations reached
  - Satisfactory fitness value achieved
  - No significant improvement in population fitness

  If the condition is satisfied, the algorithm terminates and returns the best solution found. But if the stop condition is not satisfied, the new generation replaces the old population and the process repeats.

Terms in Genetic Algorithms

- **Individual**
  A single candidate solution to the problem.
- **Population**
  A group of individuals representing multiple candidate solutions.
- **Fitness**
  A measure of how good a solution is according to the objective function.
- **Trait (Gene)**
  A specific feature or component of an individual solution.
- **Genome (Chromosome)**
  The complete set of genes that represent a candidate solution.

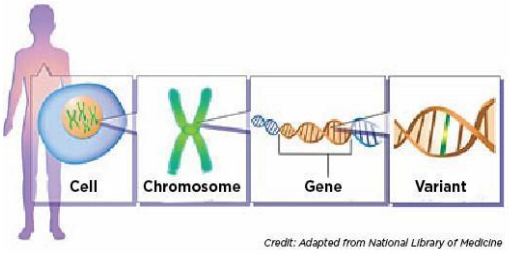
Population

In Genetic Algorithms, the concept of population is derived from biological evolution. Just like in biology where a population consists of many organisms, in Genetic Algorithms a population consists of multiple candidate solutions.

Biological Inspiration

Genetic Algorithms are inspired by biological concepts such as:
In biology:

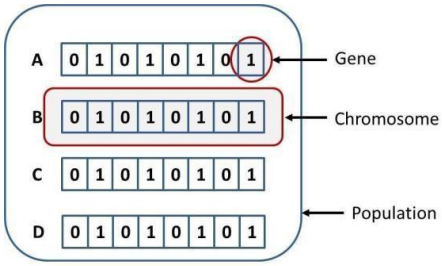
- Cells contain chromosomes
- Chromosomes contain genes
- Genes determine characteristics of an organism



Credit: Adapted from National Library of Medicine

Similarly, in Genetic Algorithms:

- A population contains multiple chromosomes
- Each chromosome represents a possible solution
- Each gene represents a variable or feature of that solution



Gene, Chromosomes and Population

Biology	**Genetic Algorithm**
Cell	Individual Solution
Chromosome	Representation of a solution
Gene	Element of a solution
Population	Set of possible solutions

Role of Population in Genetic Algorithms

The population plays a crucial role because:

- It provides diversity of solutions
- It allows the algorithm to explore multiple possibilities
- Better individuals can be selected and evolved

During each generation:

- Fitness of individuals is evaluated.
- Better individuals are selected.
- Crossover and mutation generate new individuals.
- A new population is formed.

This process continues until an optimal solution is found.

Fitness and Selection in Genetic Algorithms

In Genetic Algorithms, the concepts of fitness and selection are inspired by the biological principle of natural selection. The idea is often referred to as “Survival of the Fittest.”

Individuals that are better adapted to the environment have a higher chance of surviving and passing their genes to the next generation.

Similarly, in Genetic Algorithms:

- Individuals with higher fitness values are more likely to be selected.
- These individuals produce offspring for the next generation.
- Over multiple generations, the population gradually evolves toward better solutions.

Fitness Function

A fitness function evaluates how good a candidate solution is and measures how close a solution is to the desired or optimal result.

Fitness = Quality of the solution
Higher fitness → Better solution

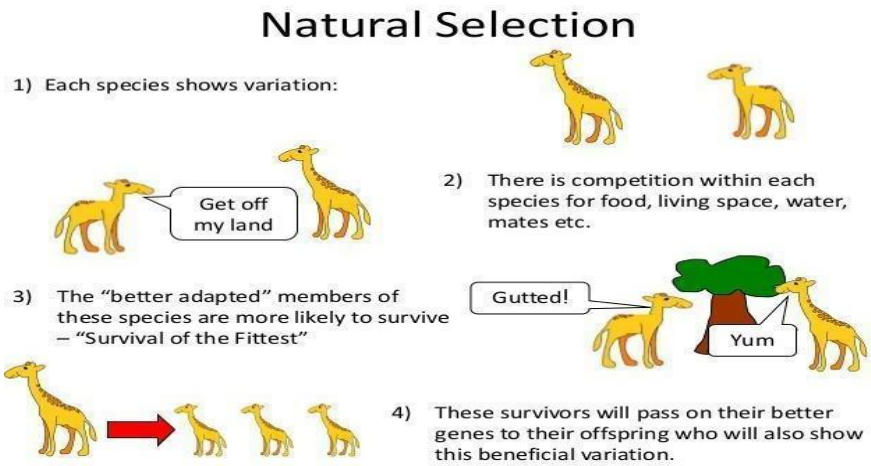
For example, in an optimization problem:

$$f(x) = -(x-3)^2 + 9$$

The goal is to find the value of x that maximizes the function. Solutions that produce higher values of f(x) will have higher fitness.

Selection

Selection is the process of choosing the best individuals from the population to create the next generation.



The purpose of selection is to ensure that:

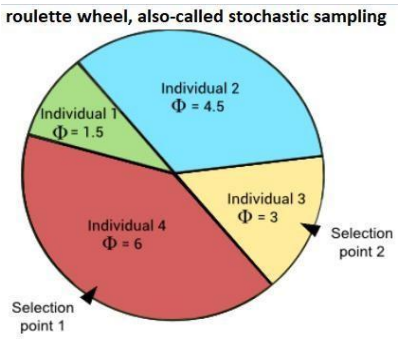
- Good solutions are preserved.
- Poor solutions gradually disappear.
- The population improves over time.

Common Selection Methods

Roulette Wheel Selection (Stochastic Sampling)

In Roulette Wheel Selection, the probability of selecting an individual is proportional to its fitness value. Individuals with higher fitness occupy larger portions of the wheel, giving them a higher chance of being selected.

Individual	**Fitness**
Individual 1	1.5
Individual 2	4.5
Individual 3	3.0
Individual 4	6.0



Here *Individual 4* has the highest fitness and therefore the highest probability of being selected.

Rank Selection

In Rank Selection, individuals are ranked based on their fitness.

Individual	**Fitness**	**Rank**
Individual 1	1.5	1
Individual 3	3.0	2
Individual 2	4.5	3
Individual 4	6.0	4

Selection probability is assigned based on the rank rather than the actual fitness value. This method helps prevent dominant individuals from overwhelming the population too early.

Role of Fitness and Selection in Genetic Algorithms

Fitness and selection help Genetic Algorithms to:

- Identify high-quality solutions
- Guide the search toward better regions of the search space
- Improve the population over generations

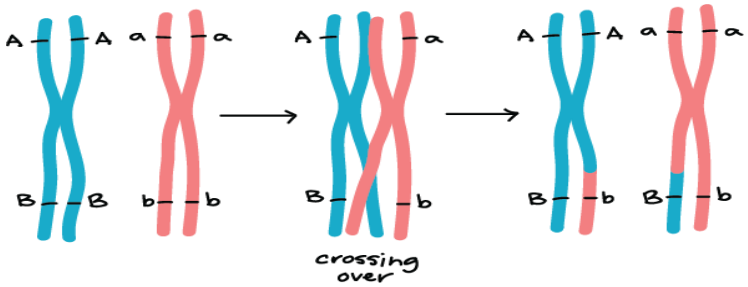
This process allows Genetic Algorithms to gradually converge toward an optimal or near-optimal solution.

Mating / Crossover / Generating Generations

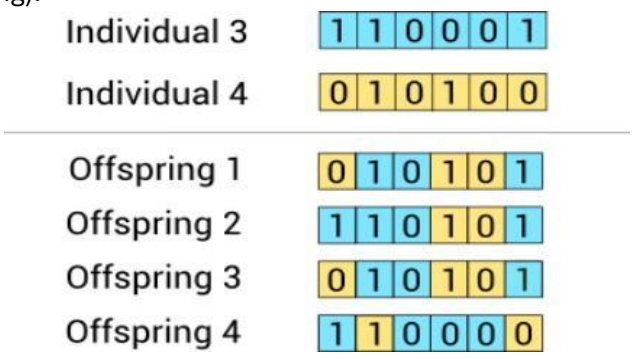
In Genetic Algorithms, the concept of mating (or reproduction) is inspired by biological reproduction. In biology, offspring inherit characteristics from their parents. Similarly, in Genetic Algorithms, new solutions are generated by combining information from selected parent solutions. This process is called Crossover or Recombination.

Biological Inspiration

In nature, organisms reproduce and pass their genes to their offspring.



In Genetic Algorithms, selected chromosomes exchange genetic information to produce new chromosomes (offspring).



Crossover Operation

Crossover combines genes from two parent chromosomes to produce new offspring.

Parent Chromosomes

Parent 1 : 110001
Parent 2 : 010100

If a crossover point is selected:

Parent 1 : 110 | 001
Parent 2 : 010 | 100

Offspring Generated

Offspring 1 : 110100
Offspring 2 : 010001

Each offspring inherits genes from both parents.

Purpose of Crossover

Crossover helps the Genetic Algorithm to:

- Combine good features from different solutions
- Explore new areas in the search space
- Generate better solutions over generations

Generating New Generations

The overall process for generating new generations is:

- Evaluate fitness of all individuals.
- Select parents based on fitness.
- Apply crossover to produce offspring.
- Apply mutation (small random changes).
- Form a new population (next generation).

This process repeats for multiple generations until:

- A satisfactory solution is found, or
- A maximum number of generations is reached.

Types of Crossover

Common crossover techniques include:

- *Single-Point Crossover*
  One crossover point divides the chromosome into two parts.
- *Two-Point Crossover*
  Two crossover points are selected, and the segment between them is exchanged.
- *Uniform Crossover*
  Genes are exchanged randomly between parents.

Role of Crossover in Genetic Algorithms

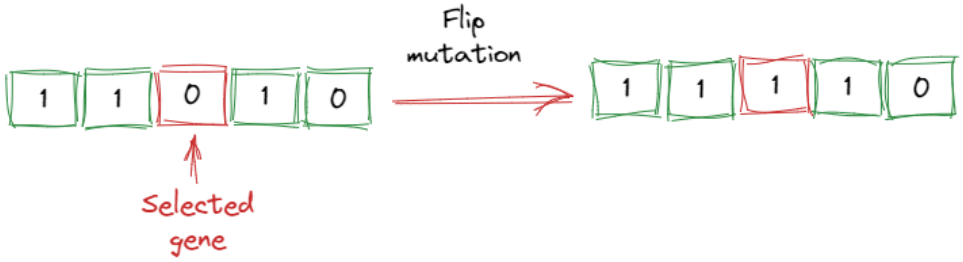
Crossover is one of the most important operations in Genetic Algorithms because it allows the algorithm to:

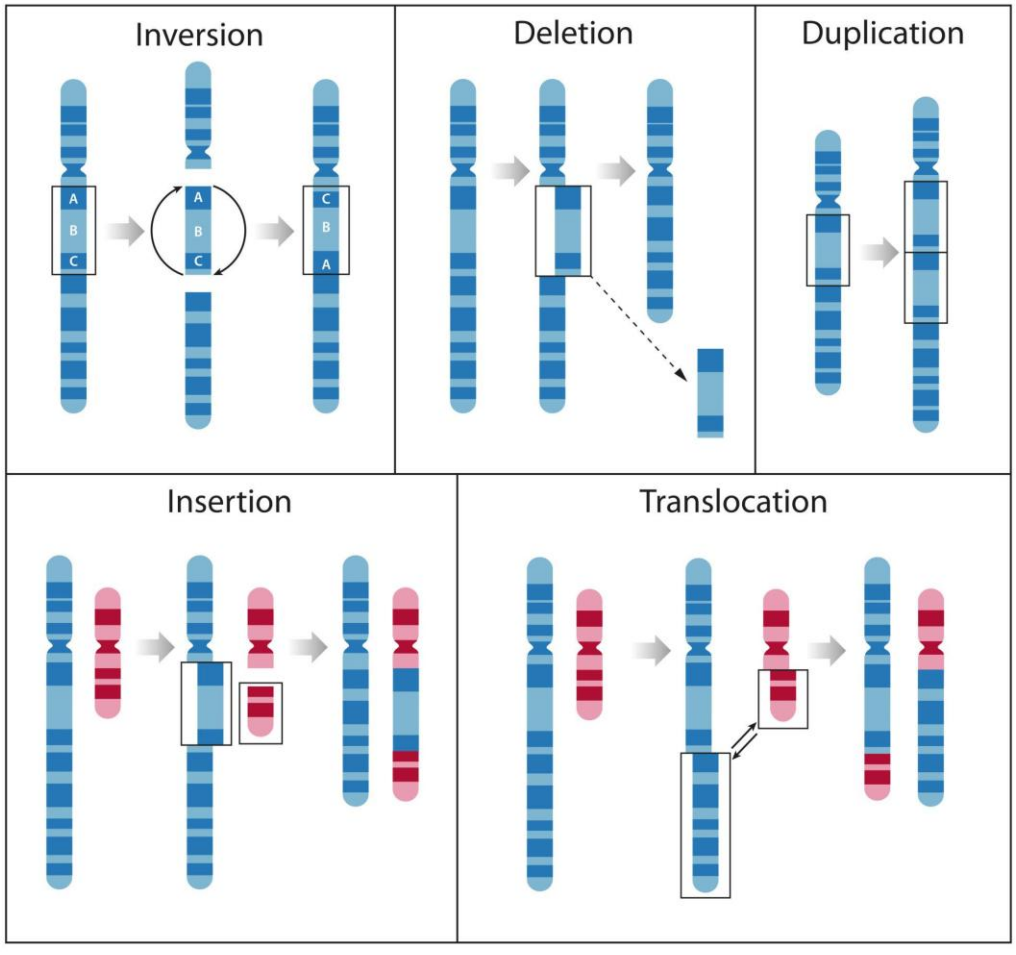
- Mix good genetic information
- Create diverse offspring
- Accelerate convergence toward better solutions

Mutation in Genetic Algorithms

Mutation is a genetic operator used to introduce small random changes in a chromosome. It is inspired by biological mutation, where small changes occur in genes during reproduction.

Mutation helps maintain genetic diversity in the population and prevents the algorithm from getting stuck in local optima.





Biological Inspiration

In nature, mutations occur naturally and introduce new genetic traits. Similarly, in Genetic Algorithms, mutation slightly modifies chromosomes to explore new solutions.

Mutation Operation

Mutation usually modifies one or more genes in a chromosome.

Original Chromosome

010101

Mutated Chromosome

010001

Here, one gene changed from:

1 → 0

This type of mutation is called Bit Flip Mutation.

Mutation Process

The mutation process works as follows:

- A chromosome is selected from the population.
- A gene position is randomly selected.
- The gene value is modified.
- The mutated chromosome becomes part of the new population.

Purpose of Mutation

Mutation helps the algorithm to:

- Maintain population diversity
- Explore new regions of the search space
- Prevent premature convergence
- Escape local maxima

Without mutation, the population may become too similar, which can reduce the ability of the algorithm to find better solutions.

Mutation Rate

Mutation does not occur for every chromosome. Instead, it happens with a small probability, called the mutation rate.

Typical mutation rate:

1% – 5%

A very high mutation rate can make the algorithm behave like a random search, while a very low mutation rate may reduce diversity.

Role of Mutation in Genetic Algorithms

Mutation is important because it:

- Introduces new genetic material
- Helps avoid stagnation in the population
- Improves the chances of finding the global optimum

Together with selection and crossover, mutation allows Genetic Algorithms to evolve better solutions over multiple generations.

Activity 2 (Example):

Genetic Algorithm for Function Optimization

Step 1 — Define the Expression

```
def evaluateExpression(x, y, z):
    return 6 * x**3 + 9 * y**2 + 90 * z - 25
```

The goal is to find values of x, y, z such that:

$$6x^3 + 9y^2 + 90z = 25$$

When the expression equals 25, the function returns 0, which represents the optimal solution.

Step 2 — Generate Initial Population

Population consists of random candidate solutions.

```
import random

solutions = []

for counter in range(1000):
    solutions.append(
        (random.uniform(0,1000),
         random.uniform(0,1000),
         random.uniform(0,1000))
    )
```

Each tuple represents an individual solution.

(x, y, z)

Step 3 — Fitness Function

The closer the result is to 0, the better the solution.

```
def fitness(x, y, z):

    ans = evaluateExpression(x, y, z)

    if ans == 0:
        return 99999
    else:
        return abs(1/ans)
```

A higher fitness value indicates a better solution.

Step 4 — Selection and Mutation

Generate new populations based on best solutions.

```
for generation_count in range(10000):

    rankedSolutions = []
    for solution in solutions:
        rankedSolutions.append(
            (fitness(solution[0], solution[1], solution[2]), solution)
        )
```

```
        rankedSolutions.sort()
        rankedSolutions.reverse()

        print(f"=== Generation {generation_count} best solutions ====")
        print(rankedSolutions[0])

        if rankedSolutions[0][0] > 999:
            break

        bestSolution = rankedSolutions[:100]

        variables = []

        for solution in bestSolution:
            variables.append(solution[1][0])
            variables.append(solution[1][1])
            variables.append(solution[1][2])

        newGeneration = []

        for counter in range(1000):

            x = random.choice(variables) * random.uniform(0.99,1.01)
            y = random.choice(variables) * random.uniform(0.99,1.01)
            z = random.choice(variables) * random.uniform(0.99,1.01)

            newGeneration.append((x,y,z))

        solutions = newGeneration
```

Note: Here is the link for the Jupyter Notebook (.ipynb) for the above example.

Activity 3 (Problem Statements / Tasks):

1. *Genetic Algorithm for Expression Optimization*

   Consider the following expression:

   $$6x^3 + 9y^2 + 90z = 25$$

   Requirements

   - Implement the Genetic Algorithm to find suitable values of:
     x, y, z
   - Generate an initial population of 1000 individuals.
   - Implement:
     - fitness function
     - selection of best individuals
     - mutation

   Display

   - Best solution in each generation
   - Final optimized values of x, y, z
   - Fitness value

2. *Genetic Algorithm Population Analysis*

Modify the Genetic Algorithm implementation to observe population evolution.

Requirements

Display for each generation:

- Best fitness value
- Worst fitness value
- Average fitness value

Explain

- How population improves across generations.

3. *Mutation Impact Experiment*

Modify the mutation range.

Original mutation:

random.uniform(0.99,1.01)

Test with:

random.uniform(0.9,1.1)

Requirements

- Run the algorithm with both mutation ranges.

Compare:

- Number of generations required
- Best fitness obtained

Explain:

- How mutation affects convergence.

4. *Genetic Algorithm for Maximizing a Function*

Consider the function:

$$f(x) = -x^2 + 10x + 5$$

Requirements

- Use a Genetic Algorithm to find the value of x that maximizes the function.

Steps:

- Generate random population of x values.
- Define fitness function as f(x).
- Apply selection and mutation.

Display

- Best value of x
- Maximum value of f(x)
- Number of generations required

5. *Genetic Algorithm for the Knapsack Problem*

Consider a knapsack with capacity:

50 units

Items:

Item	Weight	Value
A	10	60
B	20	100
C	30	120

Tasks

- Explain how a Genetic Algorithm can be used to solve the Knapsack problem.

Describe:

- Representation of chromosomes
- Fitness function
- Selection
- Mutation

6. *8-Queens Problem using Genetic Algorithm*

The 8-Queens problem requires placing 8 queens on an 8×8 chessboard such that no two queens attack each other.

A queen can attack another queen if they are in the same:

- row
- column
- diagonal

In Genetic Algorithms, a possible board configuration can be represented as a chromosome.

Example chromosome representation:

[4, 2, 7, 3, 6, 8, 5, 1]

Where:

Index → column number

Value → row position of the queen

This means:

Column 1 → Row 4

Column 2 → Row 2

Column 3 → Row 7

...

Requirements

- Generate an initial population of random chromosomes.
  Example:
  [3,5,2,8,1,4,7,6]
  [4,2,7,3,6,8,5,1]
  [2,6,1,7,4,8,3,5]
- Implement a fitness function that counts the number of non-attacking queen pairs.
  Maximum fitness occurs when:
  28 non-attacking pairs
- Implement the following Genetic Algorithm steps:
  - Selection of best chromosomes
  - Crossover between two parents
  - Mutation (random change in queen position)
- Generate new generations until:
  - a valid solution is found
  - or a maximum number of generations is reached

Display

Your program should display:

- Best chromosome in each generation
- Fitness value of best chromosome
- Number of generations required

- Final board configuration

Example Output

Solution Found:
[4, 2, 7, 3, 6, 8, 5, 1]
Fitness = 28

This represents a valid placement of 8 queens with no conflicts.

Conceptual Questions

Students should also explain:

- Why Genetic Algorithms are suitable for the 8-Queens problem.
- What happens if mutation is removed.
- How population size affects the search performance.